

EDS Lab Assignments

Practical 1

Name: Swamini Bhagwat

PRN: 202501100023

1.1.1 Calculate Momentum

Problem Statement:

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula: $p = m \times v$

where:

m is the mass of the object (in kilograms).

v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Code:

```
mass = float(input())
velocity = float(input())
momentum = mass * velocity
print(f"{momentum:.2f} kgm/s")
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement for '1.1.1. Calculate Momentum' is visible, including the formula $p = m \times v$ and the input/output requirements. The main editor shows the Python code for calculating momentum. The right-hand panel displays the execution results, indicating that 2 out of 2 shown test cases and 2 out of 2 hidden test cases passed. The test cases show expected and actual outputs for mass and velocity inputs, resulting in momentum values rounded to two decimal places.

Test Case	Expected Output	Actual Output
Test case 1	5.0 10.0 50.00kg/s	5.0 10.0 50.00kg/s
Test case 2	2.3 4.8 11.04kg/s	2.3 4.8 11.04kg/s

1.1.2 Conditional Calculation based on the Number of Digits

Problem Statement:

Write a Python program that accepts an integer as input. Depending on the number of digits in.

Constraints: $1 \leq n \leq 999$

Input Format:

- The input consists of a single integer.

Output Format:

- If is a single-digit number, print its square.
- If is a two-digit number, print its square root (rounded to two decimal places).
- If is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid"

Code:

```
import math
n=int(input())
sq = n*n
sq_root = math.sqrt(n)
cb_root = math.cbrt(n)
if n>1 and n<=9:
    print(f"{sq}")
elif n>=10 and n<=99:
    print(f"{sq_root:.2f}")
elif n>=100 and n<=999:
    print(f"{cb_root:.2f}")
else:
    print("Invalid")
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement and constraints are visible. The main editor shows the Python code for conditional calculation based on the number of digits. The right sidebar shows the test results, indicating that 4 out of 4 shown test cases passed and 3 out of 3 hidden test cases passed. The test cases table shows the expected and actual outputs for two test cases.

Test Case	Expected Output	Actual Output
Test case 1	81	81
Test case 2	4.62	4.62

1.1.3 Days Between Two Dates

Problem Statement:

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format YYYY-MM-DD.
- The second line contains the second date in the format YYYY-MM-DD.

Output Format:

- An integer representing the number of days between the given dates.

Code:

```
from datetime import date
from datetime import datetime
date1 = input().strip()
date2 = input().strip()
d1 = datetime.strptime(date1, "%Y-%m-%d")
d2 = datetime.strptime(date2, "%Y-%m-%d")
difference = d2-d1
print(difference.days)
```

Output:

The screenshot displays the CodeTANTRA online IDE interface. On the left, the problem statement for '1.1.3. Days between Two Dates' is shown, including the input/output format and a note. The main editor area contains the Python code for calculating the number of days between two dates. The right sidebar shows the test results, indicating that 2 out of 2 shown test cases passed and 2 out of 2 hidden test cases passed. The test cases are detailed below:

Test Case	Expected Output	Actual Output
Test case 1	2014-07-02 2014-11-02 123	2014-07-02 2014-11-02 123
Test case 2	2014-07-02 2014-07-11 9	2014-07-02 2014-07-11 9

1.1.4 Reverse a Number

Problem Statement:

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Code:

```
n=int(input())
reverse=int(str(n)[::-1])
print(reverse)
```

Output:

The screenshot displays the CODETANTRA online coding environment. On the left, the problem statement for '1.1.4. Reverse a Number' is visible, including the input and output formats. The main editor shows a Python solution using a while loop to reverse the digits of a number. The right sidebar provides performance metrics and test case results.

Performance Metrics:

Metric	Value
Average time	0.006 s
Maximum time	0.008 s

Test Case Results:

- 2 out of 2 shown test case(s) passed
- 3 out of 3 hidden test case(s) passed

Test Case 1 Details:

Expected output	Actual output
5367	5367
7635	7635

Test Case 2 Details:

Expected output	Actual output
0132	0132
771	771

The bottom of the interface includes a terminal, test cases tab, and navigation buttons (Prev, Reset, Submit, Next).

1.1.5 Multiplication Table

Problem Statement:

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:
<number> x <multiplier> = <result>

Code:

```
n = int(input())
```

```
for i in range(1,11):
```

```
    print(n, "x", i, "=", n*i)
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement for '1.1.5. Multiplication Table' is visible, including the input/output format and a sample test case. The main editor shows the following Python code:

```
1 #write your code here...
2 n = int(input())
3 for i in range(1,11):
4     print(n,"x",i,"=",n*i)
```

Below the code editor, the execution results are shown. The average time is 0.003 s and the maximum time is 0.004 s. The output indicates that 2 out of 2 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. A detailed view of Test case 1 shows the expected output (8) and the actual output (8), with a list of multiplication results for n=8:

Expected output	Actual output
8	8
8 x 1 = 8	8 x 1 = 8
8 x 2 = 16	8 x 2 = 16
8 x 3 = 24	8 x 3 = 24
8 x 4 = 32	8 x 4 = 32
8 x 5 = 40	8 x 5 = 40
8 x 6 = 48	8 x 6 = 48
8 x 7 = 56	8 x 7 = 56
8 x 8 = 64	8 x 8 = 64
8 x 9 = 72	8 x 9 = 72
8 x 10 = 80	8 x 10 = 80

At the bottom, there are buttons for 'Terminal', 'Test cases', 'Prev', 'Reset', 'Submit', and 'Next'.

1.2.1 Pass or Fail

Problem Statement:.

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer, the number of courses.
- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Code:

```
def calculate(marks, total_courses):
    if any(mark<40 for mark in marks):
        return "Fail"
    total_marks = sum(marks)
    aggregate_percentage = ((total_marks)/(total_courses*100))*100
    if aggregate_percentage > 75:
        grade = "Distinction"
    elif aggregate_percentage >= 60:
        grade = "First Division"
    elif aggregate_percentage >= 50:
        grade = "Second Division"
    elif aggregate_percentage >= 40:
        grade = "Third Division"
    return (aggregate_percentage, grade)
num_courses = int(input())
marks = list(map(int,input().split()))
result = calculate(marks, num_courses)
if result == 'Fail':
    print("Fail")
else:
```

```

aggregate_percentage, grade = result
print("Aggregate Percentage:",f"{aggregate_percentage:.2f}")
print(f"Grade: {grade}")

```

Output:

The screenshot displays the CODETANTRA IDE interface. On the left, the problem statement for '1.2.1. Pass or Fail' is visible, detailing the logic for determining a student's grade based on their aggregate percentage across multiple courses. The logic is as follows:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

The program also includes input and output formats, and a note about the aggregate percentage calculation.

On the right, the Python code is shown in the editor. It defines a function `cal` that takes marks and total courses as input and returns the grade. The main code reads the number of courses and marks, calculates the aggregate percentage, and prints the result.

At the bottom, the test cases are displayed, showing that 5 out of 5 shown test cases passed. The test cases are as follows:

Test Case	Expected Output	Actual Output
Test case 1	4 55 55 78 88 Aggregate Percentage: 71.75 Grade: First Division	4 55 55 78 88 Aggregate Percentage: 71.75 Grade: First Division
Test case 2	5 56 76 97 86 99 Aggregate Percentage: 82.00 Grade: Distinction	5 56 76 97 86 99 Aggregate Percentage: 82.00 Grade: Distinction
Test case 3	5 99 85 43 37 34 Fail	5 99 85 43 37 34 Fail
Test case 4	Expected output	Actual output

1.2.2 Fibonacci Series

Problem Statement:

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

- A single integer representing the number of terms to generate.

Output Format:

- A single line containing the first terms of the Fibonacci sequence, separated by spaces.

Code:

```
def fibonacci(n, a=0, b=1):  
    if n==0:  
        return a  
    else:  
        return fibonacci(n-1,b,a+b)
```

```
n=int(input())  
for i in range(n):  
    print(fibonacci(i),end=" ")
```

Output:

The screenshot displays a coding interface for the '1.2.2. Fibonacci Series' problem. The left sidebar contains the problem description and input/output formats. The main editor shows a Python solution using recursion. The right sidebar displays test case results, indicating that 2 out of 2 shown test cases and 2 out of 2 hidden test cases passed. The test cases show the expected output and actual output for different input values.

Problem Description: Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format: A single integer n representing the number of terms to generate.

Output Format: A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases:

Test Case	Expected Output	Actual Output
Test case 1	0 1 1 2 3	0 1 1 2 3
Test case 2	0 1 1 2 3 5 8 13 21 34 55 89 144 233 377	0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

1.2.3 Pattern-1

Problem Statement:

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Code:

```
rows = int(input())
for i in range(1, rows+1):
    for j in range(i):
        print("*",end = "")
    print()
```

Output:

The screenshot displays a coding environment with the following components:

- Header:** CODETANTRA logo, navigation links (Home, Learn Anywhere), user ID (202501100023@mitaoe.ac.in), and a Logout button.
- Problem Statement:** "1.2.3. Pattern - 1" with a description: "Write a Python program to print a pattern of asterisks in the form of a right-angled triangle." It includes input and output format instructions.
- Code Editor:** A Python script named "rightangl..." is shown, implementing the pattern logic using nested loops.
- Test Results:** A summary bar indicates "2 out of 2 shown test case(s) passed" and "4 out of 4 hidden test case(s) passed". Below this, two test cases are detailed, each showing "Expected output" and "Actual output" as a right-angled triangle of asterisks.
- Footer:** A navigation bar with buttons for "Prev", "Reset", "Submit", and "Next".

1.2.4 Pattern-2

Problem Statement:

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Code:

```
n=int(input())
for i in range(1, n+1):
    for j in range(1, i+1):
        print(j,end=" ")
    print()
```

Output:

The screenshot displays a coding platform interface with the following components:

- Header:** CODETANTRA logo, navigation links (Home, Learn Anywhere), user profile (202501100023@mitaoe.ac.in), and a Logout button.
- Problem Statement:** "1.2.4. Pattern - 2" with a description: "Write a Python program to print a right-angled triangle pattern of numbers."
- Input Format:** "The input is an integer, representing the number of rows in the pattern."
- Output Format:** "The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number."
- Note:** "Refer to the displayed test cases for the sample pattern."
- Sample Test Cases:** A table showing the expected output for inputs 1 through 5.
- Code Editor:** A Python program that takes an integer input and prints a right-angled triangle pattern of numbers.
- Test Results:** A section showing the execution of two test cases, both of which passed. The results include the expected output, actual output, and execution time.

Test Case	Expected output	Actual output
Test case 1	1	1
Test case 2	1 1 2	1 1 2
Test case 3	1 1 2 1 2 3	1 1 2 1 2 3
Test case 4	1 1 2 1 2 3 1 2 3 4	1 1 2 1 2 3 1 2 3 4
Test case 5	1 1 2 1 2 3 1 2 3 4 1 2 3 4 5	1 1 2 1 2 3 1 2 3 4 1 2 3 4 5